

Box Detection Algorithm

Weaknesses & Improvement Suggestions

1. Overview

The `box_detection_solution.py` script follows a two-stage RANSAC pipeline to detect a box placed on a floor using Kinect depth data. While the approach works well under controlled conditions, it relies on several simplifying assumptions that can reduce its robustness when the scene becomes more complex or deviates from the expected setup. The pipeline proceeds through a sequence of steps:

it first loads amplitude, distance, and point cloud data from `.mat` files;

then applies RANSAC followed by an SVD-based refinement to estimate the floor plane;

performs morphological operations to clean up the floor mask;

and runs a second RANSAC step on the remaining (non-floor) points to detect the top surface of the box.

Afterward, it extracts the largest connected component corresponding to the box top and estimates its dimensions—length and width using PCA-based bounding boxes, and height via point-to-plane distance.

2. Summary of Weaknesses

#	Weakness	Suggested Improvement
W1	Hard-coded thresholds tied to the example index	Derive the threshold from the sensor noise model or local point density instead
W2	Fixed RANSAC iteration count	Switch to adaptive stopping based on the estimated inlier ratio (target $p = 99.9\%$)
W3	RANSAC scoring only looks at 6,000 points out of potentially 200,000+	Switch to spatially stratified sampling to get more representative coverage
W4	SVD refit only runs once after RANSAC finishes	Loop: refit $\rightarrow$ recompute inliers $\rightarrow$ refit again, until the inlier set stabilizes
W5	Assumes exactly one box in the scene	Keep all components above a minimum size; let downstream logic pick the right one
W6	The second dominant plane is blindly treated as the box top	Check normal parallelism with the floor and confirm the height offset is in a sensible range
W7	PCA bounding box tends to overestimate dimensions due to noisy edges	Fit a minimum-area rectangle to the convex hull instead (rotating calipers / <code>cv2.minAreaRect</code>)
W8	Height estimate uses the mean, which is pulled down by noisy boundary points	Switch to the median (or the peak of a fitted Gaussian) to be more robust to outliers
W9	Morphological kernel sizes are fixed in pixels regardless of depth or resolution	Compute kernel size from a fixed physical radius using the focal length and mean plane depth
W10	Each frame is processed in isolation — no use of information from neighboring frames	Average plane parameters across frames (or use a Kalman filter) to reduce per-frame noise

3. Detailed Discussion

W1 — Hard-coded per-example thresholds

Right now, the thresholds are set based on which example is being run:

```
floor_threshold = 0.02 if example_idx == 1 else 0.03
```

This works for the given examples, but it does not generalize. When applying the method to a new scene, the values need to be manually tuned again. There is also no clear justification for choosing 0.02 versus 0.03, as these values are empirically selected. Ideally, the threshold should be derived from the data rather than manually defined.

How to solve this issue

We can use the known noise characteristics of the Kinect sensor (approximately $\sigma \propto z^2$) and set the threshold to 3σ , which provides a principled and scene-independent value. Alternatively, we can perform a preliminary RANSAC pass, estimate the residual distribution of inliers, and set the threshold to 2–3 standard deviations. This removes manual tuning while adding only minimal computational overhead.

W2 — Fixed RANSAC iteration count

The code currently uses a fixed number of iterations:

```
ransac_plane(..., max_iterations=300, ...)
```

While 300 iterations may be sufficient in some cases, it may be inadequate when the inlier ratio is low (e.g., cluttered scenes). Conversely, in simple scenes, many iterations are unnecessary.

How to solve this issue

We can update the inlier ratio estimate after each iteration:

$$\hat{\epsilon} = \text{best_count} / n_{\text{eval}}$$

Then, we recompute the required number of iterations:

$$N = \log(1-p) / \log(1-\hat{\epsilon}^3), \text{ with } p = 0.999$$

Once N becomes smaller than the number of iterations already performed, the algorithm can terminate early. This adaptive RANSAC strategy improves both efficiency and robustness.

W3 — Small random subsample for scoring

Using a random subset of 6,000 points improves efficiency but introduces bias. Kinect data is not uniformly distributed; central regions are denser and more reliable. As a result, random sampling over-represents central regions and under-represents edges, which may lead to biased scoring.

How to solve this issue

We can apply spatially stratified sampling by dividing the image into a grid and sampling proportionally from each region. This ensures better spatial coverage. Alternatively, voxel-based down sampling (e.g., 1 cm resolution) can be used to obtain a uniform geometric representation independent of pixel density.

W4 — SVD refit performed only once

After RANSAC identifies a plane, a single SVD refit is performed. However, this refitting slightly changes the plane, meaning some points may newly become inliers or outliers. Since this is not updated, the result is not fully optimized.

How to solve this issue

We can iteratively refine the plane by repeating refit → recompute inliers → refit. The process stops when the inlier set stabilizes. This typically converges within a few iterations and improves accuracy.

W5 — Assumes exactly one box

The connected component step selects only the largest region. This assumption fails when multiple objects of similar size exist or when a larger object is present.

How to solve this issue

We can return all components above a minimum size threshold instead of selecting only one. A higher-level decision process can then identify the relevant object. Additionally, simple geometric constraints such as aspect ratio or footprint size can be used to filter out unlikely candidates.

W6 — Box top assumed to be second dominant plane

After removing the floor, the next detected plane is assumed to be the box top. However, other structures such as walls or tables may produce stronger plane fits.

How to solve this issue

We can validate the detected plane by checking:

- The normal is approximately parallel to the floor (dot product $\approx \pm 1$)
- The height difference lies within a realistic range (e.g., 5 cm to 1.5 m)

If these conditions are not satisfied, the plane is rejected and the next candidate is evaluated.

W7 — PCA bounding box overestimates footprint

PCA-based bounding boxes often overestimate object size due to noise at object boundaries.

How to solve this issue

We can project the points onto the plane, compute the convex hull, and apply rotating calipers to obtain the minimum-area bounding rectangle (e.g., using `cv2.minAreaRect`). As a simpler alternative, the PCA bounding box can be slightly reduced (1–2 cm per side) to compensate for edge noise.

W8 — Height uses mean of noisy distances

Height is computed as the mean distance from box-top points to the floor. However, noisy edge points can bias the mean downward.

How to solve this issue

We can use the median instead of the mean to improve robustness. Alternatively, fitting a Gaussian distribution and using its mode provides a more stable estimate.

W9 — Fixed-pixel morphological kernels

Morphological operations use fixed kernel sizes, although pixel size varies with depth. This leads to inconsistent behavior across different distances.

How to solve this issue

We can define the kernel size in physical units (e.g., 3 cm) and convert it into pixel units using depth and camera parameters. This ensures consistent performance across varying depths.

W10 — No temporal integration across frames

Each frame is processed independently, ignoring temporal information and leading to noisier estimates.

How to solve this issue

For offline processing, we can average plane parameters across frames while maintaining consistent normal orientation.

For real-time applications, a Kalman filter can be used to smooth the estimates over time.

4. Conclusion

The overall pipeline RANSAC for plane detection, SVD for refinement, and morphological operations for cleanup is fundamentally sound. The main limitations lie in robustness and generalization rather than the core methodology.

The most impactful improvements are:

- Adaptive RANSAC stopping (W2) — improves efficiency and reliability
- Data-driven thresholds (W1) — removes manual tuning
- Iterative SVD refinement (W4) — improves plane accuracy
- Minimum-area bounding box (W7) — improves footprint estimation
- Plane validation (W6) — prevents incorrect detections

The remaining aspects (W3, W5, W8, W9, W10) are important for production-level robustness. In controlled indoor scenarios with a single object, they are less critical but addressing them improves performance in real-world environments.